

9th IPM Workshop on Particle Physics

Machine Learning Application in High Energy Physics

Meisam Ghasemi Bostanabad

School of Particles and Accelerators:

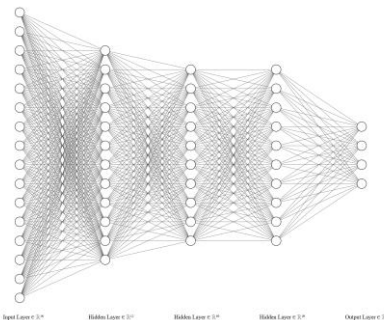
2025-12-10



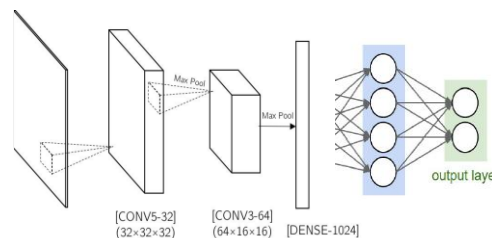
Road map

- Introduce key machine learning architectures: Deep Neural Networks (DNNs), Convolutional Neural Networks (CNNs), and Graph Neural Networks (GNN). The **training process** is covered for each model.
- Highlight the core structure and intuition behind each model.
- Demonstrate how these models are **applied in particle physics**, with real-world researches from classification of signal from background and event simulation.
- Recurrent Neural Networks (RNN), Transformers, Quantum machine learning, Unsupervised and Reinforcement learning are not covered in this talk!

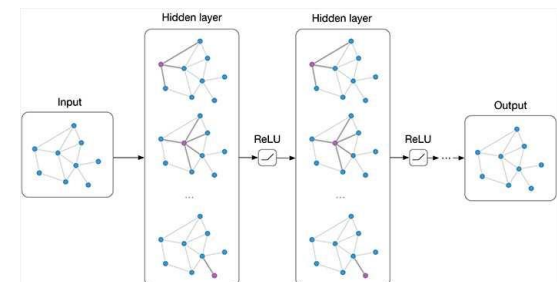
DNN



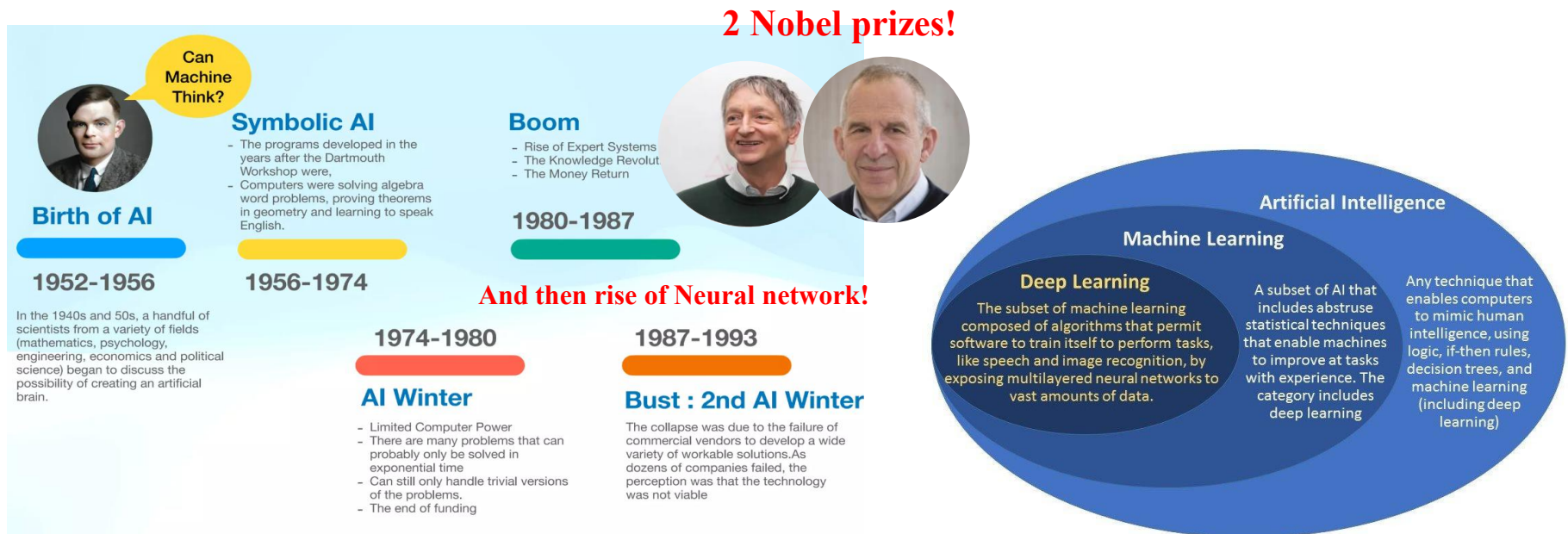
CNN



GNN

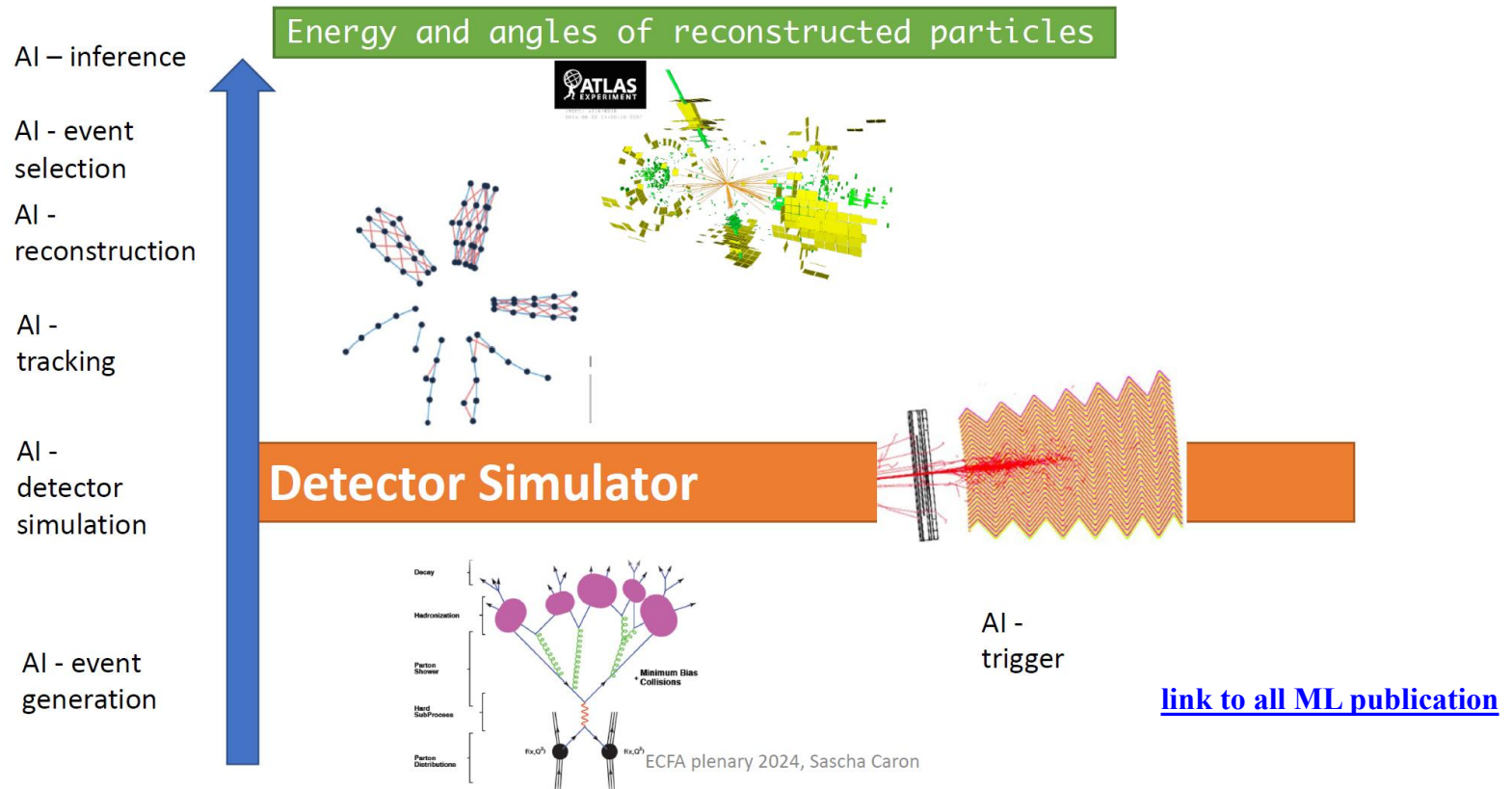


historical evolution of Artificial Intelligence



- Artificial Intelligence (AI) is the simulation of human intelligence in machines that are designed to think, learn, and make decisions.
- Machine Learning (ML) is an approach to AI where the **algorithms learn from the data.**

AI in Particle physics

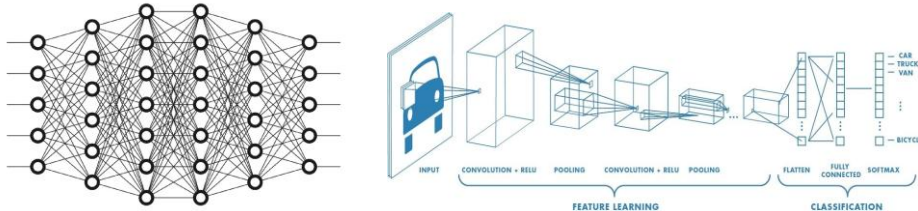


- Highlights key stages like detector simulation, tracking, and reconstruction, showcasing AI's role in enhancing ATLAS efficiency and accuracy.

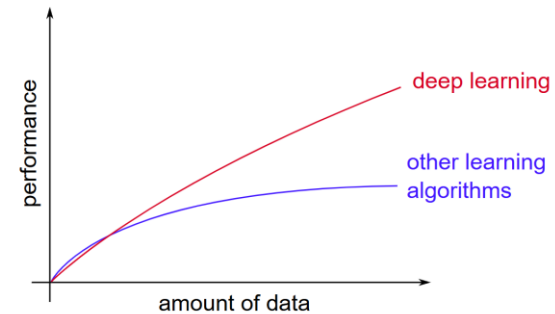
Deep Neural Network (Multi layer perceptron)

Why Deep Neural Network?

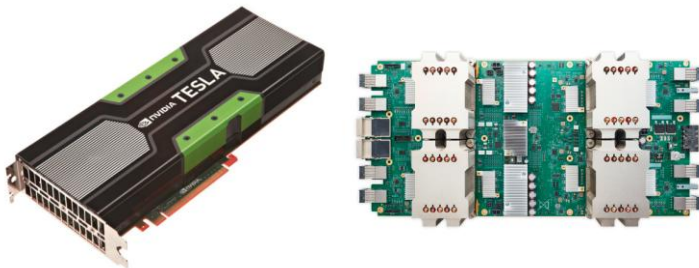
1. Better algorithm and understanding



3. Data with labels



2. Computing power (GPU and TPU)

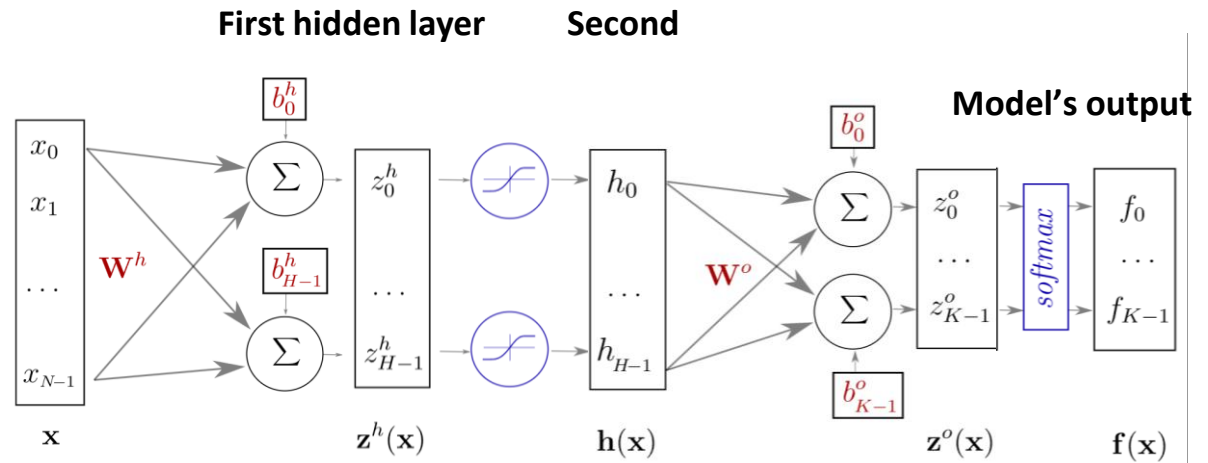


4. Open source tools and models



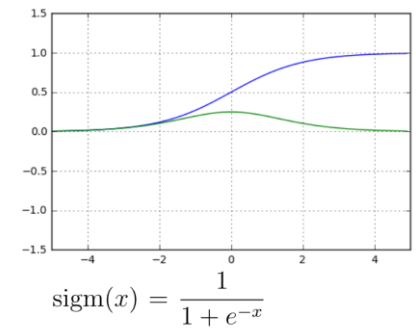
DNN and Forward Propagation

- $\mathbf{x}, f(\mathbf{x})$ input and output
- $\mathbf{z}(\mathbf{x})$ pre-activation
- $\mathbf{w}, b$ weights and bias
- g activation function



- $\mathbf{z}^h(\mathbf{x}) = \mathbf{W}^h \mathbf{x} + \mathbf{b}^h$
- $\mathbf{h}(\mathbf{x}) = g(\mathbf{z}^h(\mathbf{x})) = g(\mathbf{W}^h \mathbf{x} + \mathbf{b}^h)$
- $\mathbf{f}(\mathbf{x}) = \text{softmax}(\mathbf{z}^o) = \text{softmax}(\mathbf{W}^o \mathbf{h}(\mathbf{x}) + \mathbf{b}^o)$

Converts model outputs into probabilities



$$\text{sigm}'(x) = \text{sigm}(x)(1 - \text{sigm}(x))$$

Sigmoid activation function

Backward Propagation (SGD)

- Adjust the model's parameters θ ($W; b$) to minimize a loss (negative log likelihood).

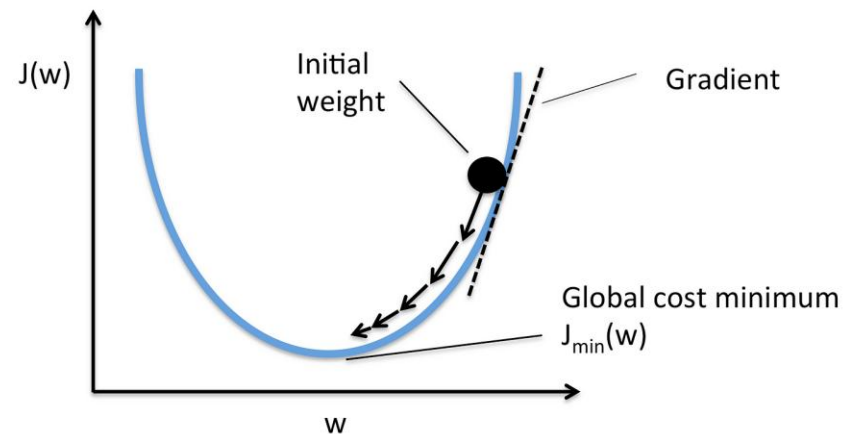
$$L_{BCE} = -\frac{1}{n} \sum_{i=1}^n (Y_i \cdot \log \hat{Y}_i + (1 - Y_i) \cdot \log (1 - \hat{Y}_i))$$

Truth labelPredicted prob

Initialize θ randomly

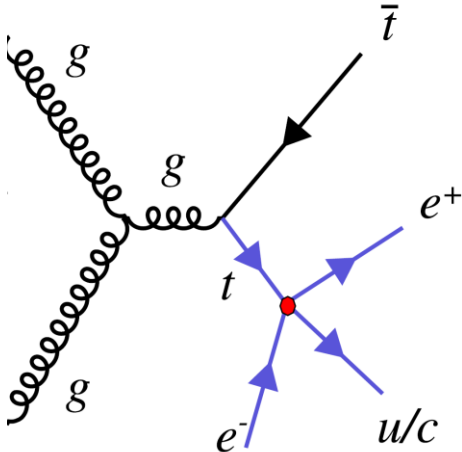
For E epochs perform:

- Randomly select a small batch of samples ($B \subset S$)
 - Compute gradients: $\Delta = \nabla_{\theta} L_B(\theta)$
 - Update parameters: $\theta \leftarrow \theta - \eta \Delta$
 - $\eta > 0$ is called the learning rate
- Repeat until the epoch is completed (all of S is covered)



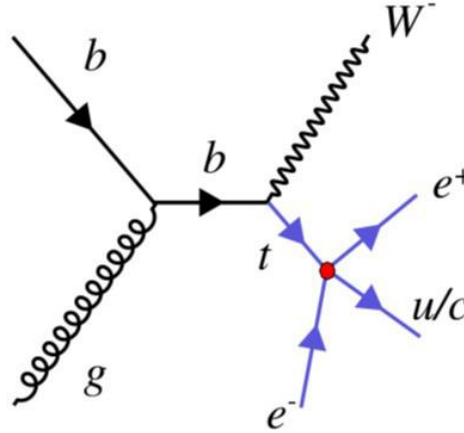
Flavor Changing in Top sector

In this analysis we looking for FC ($t \rightarrow u$ or $t \rightarrow c$) in top sector as the heaviest quark which may be an indicator of new flavor physics.



ttbar

- Starting with **ttbar**, targeting **final states** with three leptons (a pair of OP) and at least two jets with one b-tagged
- The leading potential backgrounds are $t\bar{t}, tZ, WZ, ZZ$



tW

- Next channel **tW**, targeting **final states** with three leptons (a pair of OP) and at least one light jet
- The leading potential backgrounds are tZ, WZ, ZZ

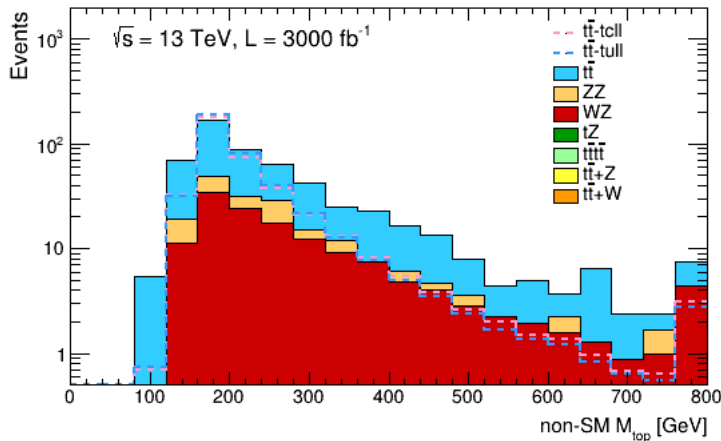
Decay rate

$$\Gamma(t \rightarrow u/c + \ell^+ + \ell^-) = \frac{m_t^5}{6144\pi^3\Lambda_\ell^4} \sum_{i,j=L,R} [|V_{ij}^\ell|^2 \mathcal{I}_V + |S_{ij}^\ell|^2 \mathcal{I}_S + |T_{ij}^\ell|^2 \mathcal{I}_T],$$

[Paper link](#)

Feature Engineering

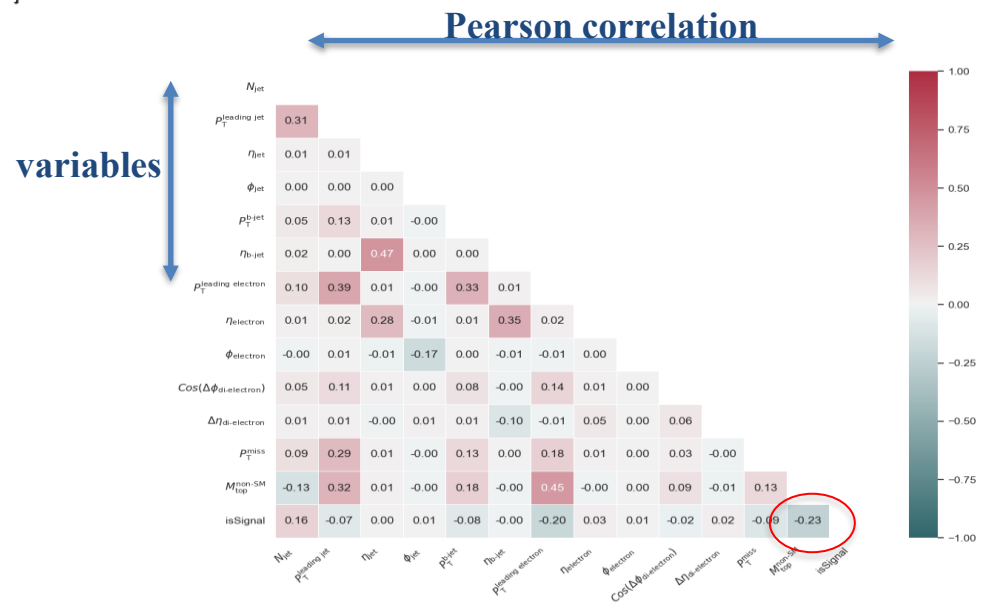
non-SM top mass distribution



The optimal algorithm combines all electrons and all non-b-tagged jets to compute the non-SM top invariant mass minimizing $|m_{ql\bar{l}} - m_{top}|$.

Peak in distribution around the top mass 174 GeV.

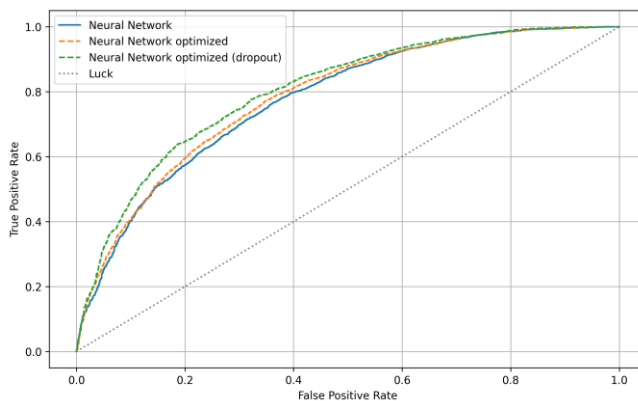
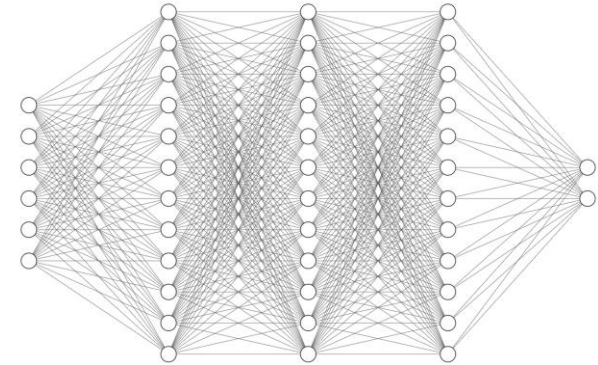
- The plot shows the correlation coefficient between variables.
- Non-SM top mass** variable has the highest correlation to the probability of being signal or background



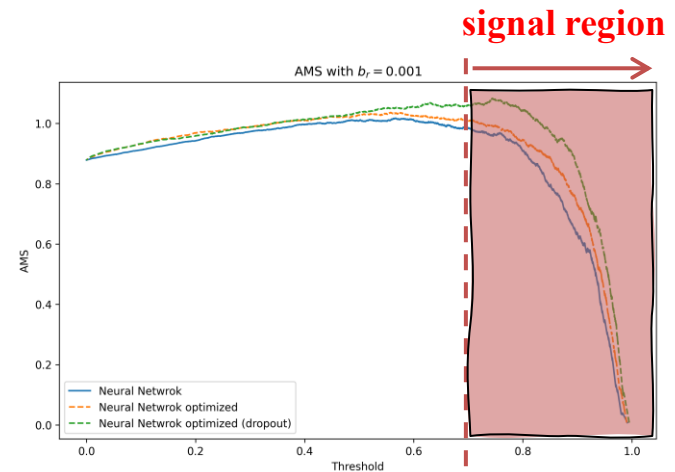
DNN Structure and Result

- DNN with 3 hidden layers each 128 neurons and sigmoid output P(sig), P(bkg). Hyperparameters are tuned and applied.
- ROC curve showing the performance of the three DNN classification models.
- The DNN threshold with the highest AMS value is the best candidate for signal region.

DNN structure



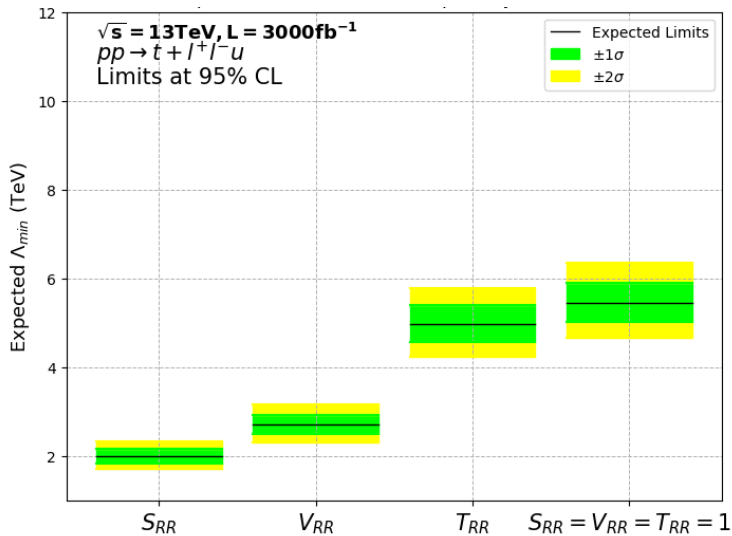
ROC curve



$$AMS = \sqrt{2 \left((TPR + FPR + b_r) \ln \left(1 + \frac{TPR}{FPR + b_r} \right) - TPR \right)}$$

Limits on NP scale

- Expected 95% exclusion limits on the new physics scale Λ for $t\bar{t}$ process shown for three coupling and mixed scenarios.
- The limits obtained using the optimized DNN alongside statistical uncertainties.
- To benchmark the performance, we compared the yields with the cut-and-count analysis using $150\text{GeV} < M_{\text{top}}^{\text{non-SM}} < 200\text{GeV}$ and $N_{\text{jet}} \geq 3$).



DNN yields

Process	NN threshold $\geq 75\%$			NN threshold $\geq 80\%$		
	$\mathcal{N}_S$	$\mathcal{N}_B$	$\mathcal{N}_S/\mathcal{N}_B$	$\mathcal{N}_S$	$\mathcal{N}_B$	$\mathcal{N}_S/\mathcal{N}_B$
$pp \rightarrow t\bar{t} \rightarrow ue^-e^+\bar{t}$	538.1	345.6	1.6	421.0	245.1	2.2

Cut-and-count yields

Process	$\mathcal{N}_S$	$\mathcal{N}_B$	$\mathcal{N}_S/\mathcal{N}_B$
$pp \rightarrow t\bar{t} \rightarrow ue^-e^+\bar{t}$	105.3	98.5	1.07

Convolutional Neural Network (CNN)

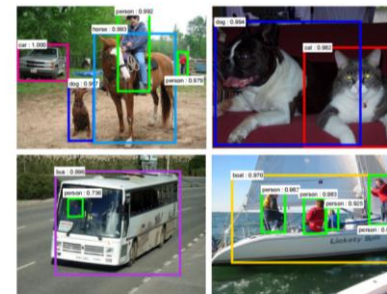
Application of CNNs (input images)

- CNN can process images and videos for further classification or generation. State of the art models like AlexNet widely used in computer vision.

Image classification



[Krizhevsky 2012]

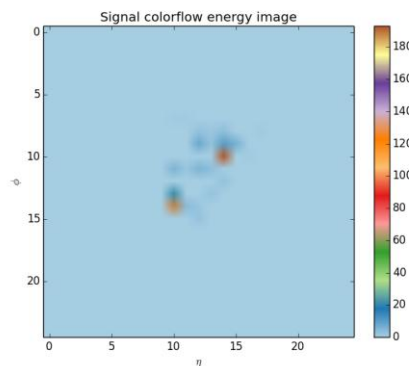


[Faster R-CNN - Ren 2015]

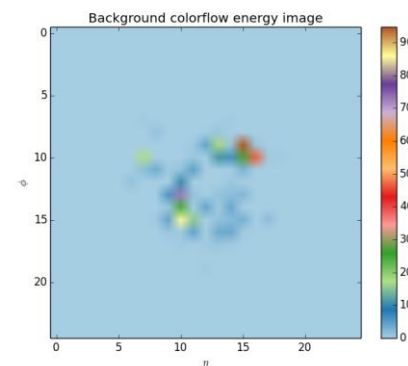
Object detection

- CNN models can be used for event classification like Higgs vs QCD by looking at jet patterns.

Higgs



QCD



Convolution and Pooling Layers

Convolutional layer

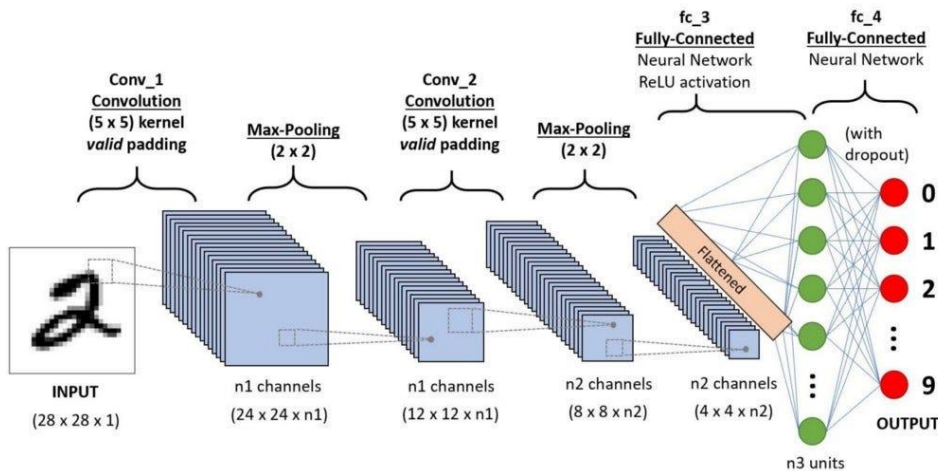
3 ₀	3 ₁	2 ₂	1	0
0 ₂	0 ₂	1 ₀	3	1
3 ₀	1 ₁	2 ₂	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

2D-convolutions (actually 2D cross-correlation):

$$(f \star g)(x, y) = \sum_n \sum_m f(n, m) \cdot g(x + n, y + m)$$

f is a convolution **kernel** or **filter** applied to the 2-d map g (our image)



Convolution layer

Applies learnable filters across the input to extract local features

Max-pool layer

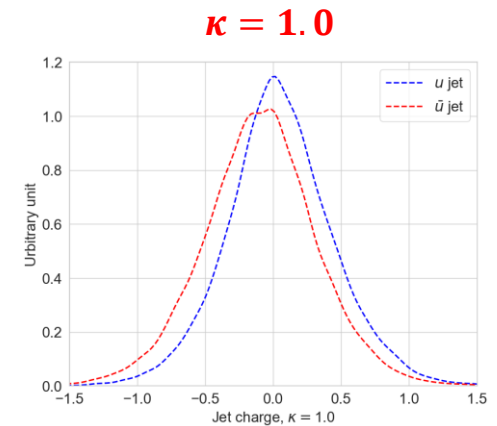
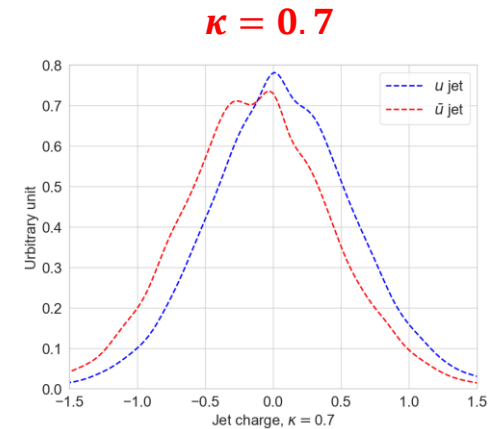
Reduces the spatial size by selecting the maximum value

Application of Neural Net to Jet charge

- The jet charge method is crucial in verifying the top quark charge, heavy boson identification (W'/Z'), quark vs. gluon jet discrimination.
- Due to trigger or analysis selections, the jet charge is not equal to the sum of jet component's charges.
- The jet charge observables are defined as (where κ is a tunable parameter affecting charge sensitivity):

$$Q_j = \frac{1}{(p_T^{\text{jet}})^\kappa} \sum_{i \in \text{Tr}} Q_i (p_T^i)^\kappa$$

$$Q_{3,\kappa} = \frac{\sum_{i \in \text{Tr}} q_i |\Delta\eta_i|^\kappa}{\sum_{i \in \text{Tr}} |\Delta\eta_i|^\kappa}$$

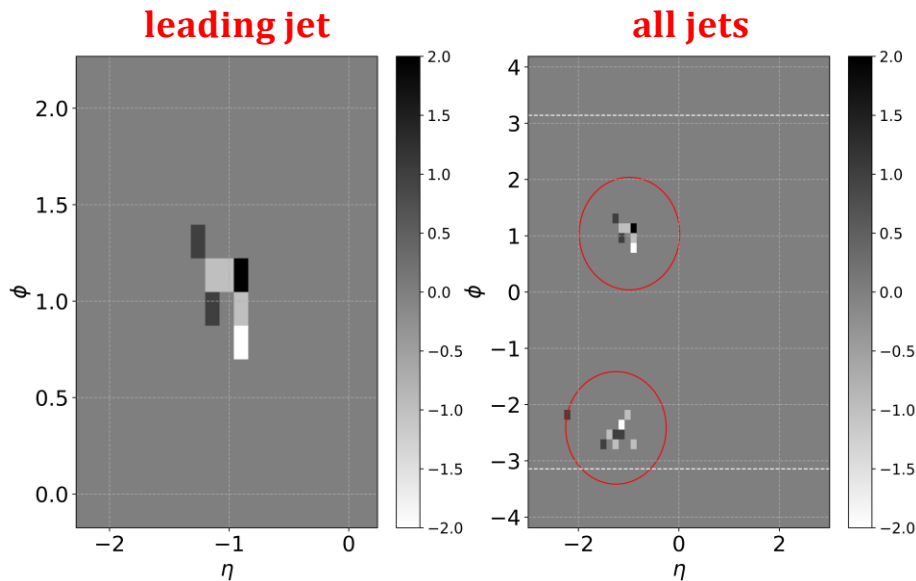


Momentum weighted jet charge distribution

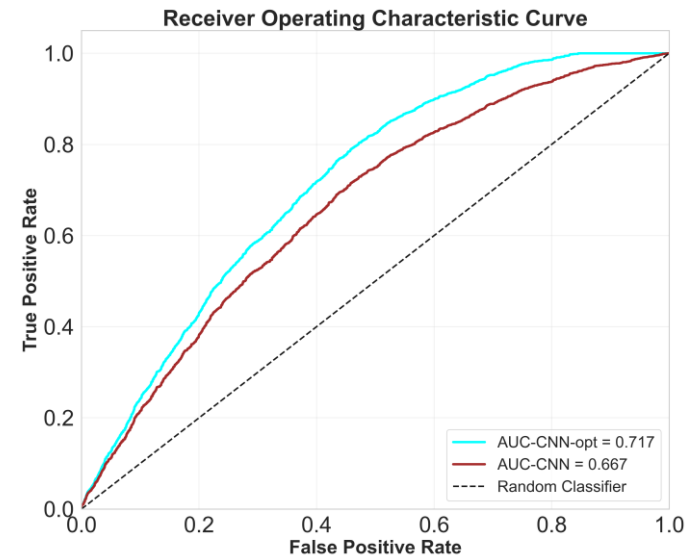
CNN Result

pixelated representation of a leading jet in the η - ϕ space, showing regions with electric charge, used as input for the CNN model.

ROC curves for baseline and optimized CNN models. The optimized model (AUC=0.717) outperforms the baseline, highlighting the benefit of hyperparameter tuning.



Pixelated representation of jets in $\eta - \phi$ space



Baseline and Optimized CNN

To improve performance, an optimized CNN was constructed using the Keras Tuner framework with randomized hyperparameter search.

TABLE I: Comparison of baseline and optimized CNN hyperparameter

Hyperparameter	Baseline CNN	Optimized CNN
<u>Conv1 filters</u>	<u>32</u>	<u>128</u>
<u>Conv2 filters</u>	<u>64</u>	<u>128</u>
Kernel size	5×5	3×3
Activation	ReLU	ReLU
Dropout (after Conv1)	30%	30%
<u>Dropout (after Dense)</u>	<u>40%</u>	<u>30%</u>
Dense layer size	64	128
Optimizer	Adam	Adam
Learning rate	10^{-4}	10^{-5} – 10^{-3}
Learning rate scheduler	None	LROnPlateau
Loss function	Binary entropy	Binary entropy

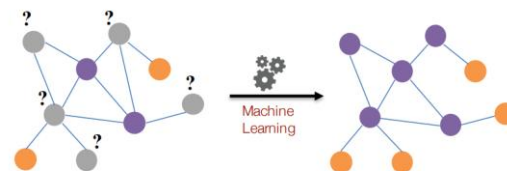
Optimized CNN with more parameters

Graph Neural Network (GNN)

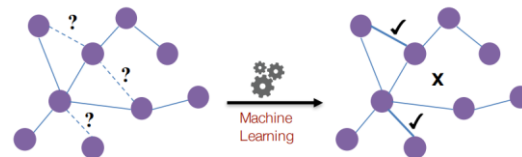
Graph NN Application

- Graph $G = (V; E)$ is a data structure consisting of two components V as nodes and E as edges.
- Node Classification: predict properties of individual nodes and label them.
- Link Prediction: infer missing or potential connections between nodes.
- Graph Classification: predict an overall property of the entire graph (e.g., classify jet as quark- or gluon-initiated).
- Unlike grid-like data, GNN excel at dealing with **non-Euclidean data**.

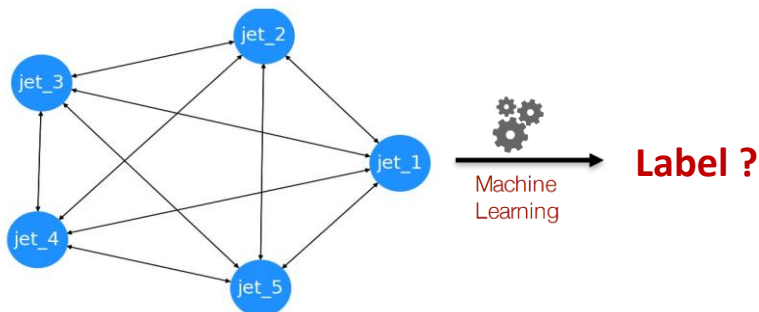
Node classification



Link prediction

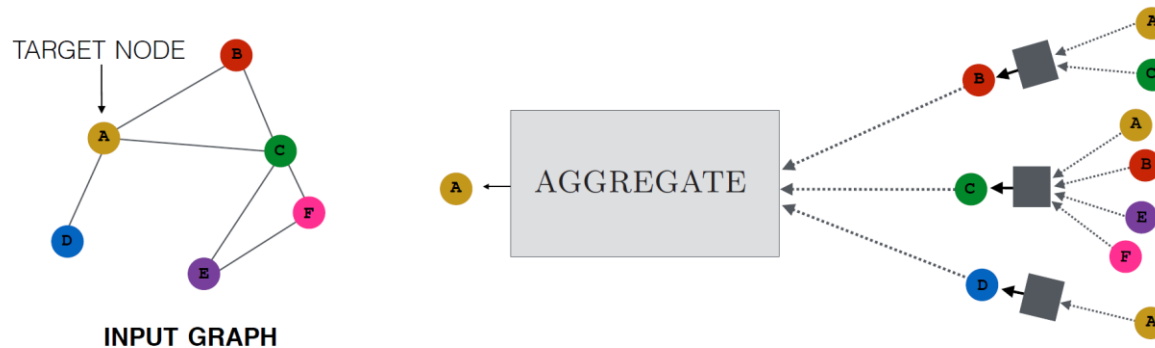


Graph classification



Graph NN Training

- GNN Training Process: Each node exchanges information with its neighbors through **message passing**, updates its embedding, and then these embeddings are aggregated to form a graph-level prediction. the model's parameters are optimized by minimizing a loss function.

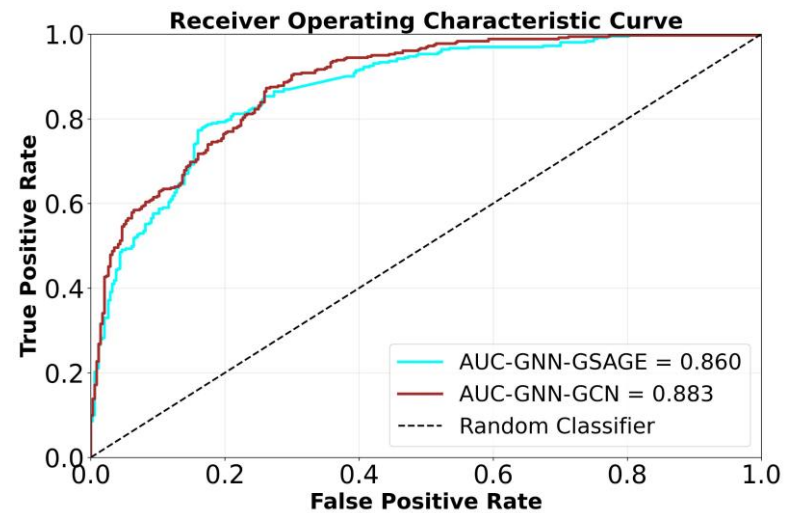
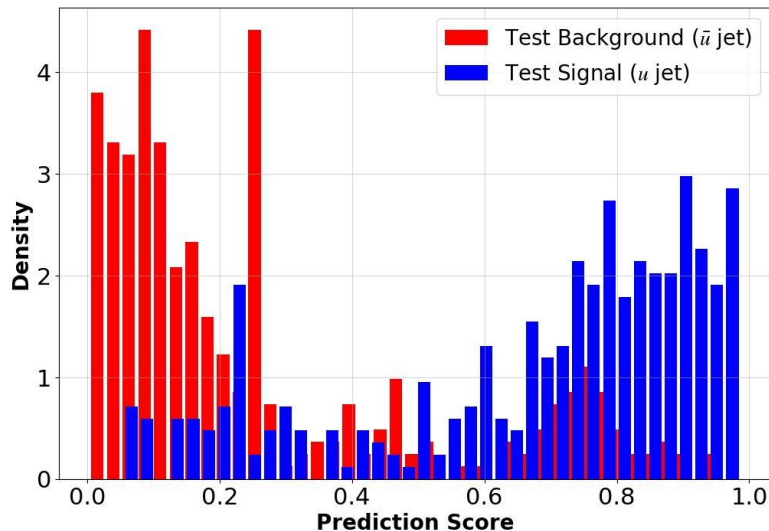


$$\mathbf{h}_u^{k+1} = \text{Update}^k (\mathbf{h}_u^k, \text{Aggregate}(\mathbf{h}_v^k \mid \forall v \in N(u)))$$

$$\mathbf{h}_u^k = \sigma \left(\mathbf{w}_{self}^k \mathbf{h}_u^{k-1} + \mathbf{w}_{neigh}^k \sum_{v \in N(u)} \mathbf{h}_v^{k-1} + \mathbf{b}^k \right)$$

Graph NN Results

- After GNN training we can compute score for all labels to see the model performance. Distributions should be well separated.
- GNN models show the best performance in terms of AUC metric. This means GNN model can find the internal structure of jets more effectively.



Summary

- Deep Neural Networks: introduced basic structure and training process using forward and backward propagation. Applied to tasks like classification of events and SUSY-background separation.
- Convolutional Neural Networks: explained how CNNs exploit spatial structure of detector data. Used in jet image analysis from Higgs boson decay and event classification.
- Graph Neural Networks: introduced message passing and graph representation learning, where particles are treated as nodes and their kinematic or spatial relations as edges.
- Quantum Machine Learning: discussed principles of qubits and quantum circuits. Highlighted potential applications in high-dimensional data and speedup of certain learning tasks.
- Neural Network Models: employed deep learning architectures in two analyses and got better result compared to traditional statistical analysis.

ML Roadmap

1. Introduction to machine learning:
 - non neural network models: [Google course](#)
 - neural network models specially DNN and CNN: [Coursera](#)
 - MIT neural network course: [link](#)
2. Graph Neural Network: Stanford [CS224 course](#)
3. Recurrent neural network: Stanford [lecture](#)
4. Generative Deep Learning: [O'reilly book](#)
5. Quantum Computation: IBM Qiskit quantum [course](#)
6. Quantum Machine Learning: IBM QML [course](#)
7. Tensorflow [Tutorial](#)
8. Pytorch [Tutorial](#)